AttendSmart GitHub Workflow Guide

Complete Step-by-Step Git Guide for Beginners

GitHub Setup (Day 1 - Krishna Only)

Krishna's Initial Repository Setup

Step 1: Create GitHub Repository

1. **Go to** GitHub.com and sign in
2. **Click "New Repository"** (green button)
3. **Repository Details:**
   - Repository name: `AttendSmart`
   - Description: `Smart Attendance System for Colleges - SIH 2025`
   - Set to **Public** (for SIH submission)
   - Check "Add a README file"
   - Add .gitignore: **React Native**
   - Choose License: **MIT License**
4. **Click "Create Repository"**

Step 2: Clone to Local Computer

```
# Navigate to your projects folder
cd Desktop

# Clone the repository
git clone https://github.com/YourUsername/AttendSmart.git

# Enter the project folder
cd AttendSmart
```

Step 3: Create Branch Structure

```
# Create and switch to development branch
git checkout -b development

# Push development branch to GitHub
git push -u origin development

# Create feature branches for team members
git checkout -b krishna-backend
git checkout -b shambavi-frontend
git checkout -b rakshitha-design
git checkout -b ananya-analytics
git checkout -b hitharth-features
git checkout -b yashas-testing
git checkout -b nuhan-ui-support

# Push all branches to GitHub
git push -u origin krishna-backend
git push -u origin shambavi-frontend
git push -u origin rakshitha-design
git push -u origin ananya-analytics
git push -u origin hitharth-features
git push -u origin yashas-testing
git push -u origin nuhan-ui-support

# Go back to development branch
git checkout development
```

Team Member Setup (Day 1 - Everyone Except Krishna)

Each Team Member's Setup Process

Step 1: Clone Repository

```
# Open terminal/command prompt
cd Desktop

# Clone the shared repository
git clone https://github.com/KrishnaUsername/AttendSmart.git

# Enter project folder
cd AttendSmart
```

Step 2: Switch to Your Personal Branch

Shambavi:

```
git checkout shambavi-frontend
```

Rakshitha:

```
git checkout rakshitha-design
```

Ananya:

```
git checkout ananya-analytics
```

Hitharth:

```
git checkout hitharth-features
```

Yashas:

```
git checkout yashas-testing
```

Nuhan:

```
git checkout nuhan-ui-support
```

Daily Git Workflow for Everyone

Before Starting Work Each Day

Step 1: Get Latest Changes

```
# Make sure you're on your branch
git checkout your-branch-name

# Get latest changes from development
git pull origin development

# If there are conflicts, Krishna will help resolve them
```

During Work - Saving Progress

Step 2: Save Your Work Frequently

```
# Check what files you've changed
git status

# Add specific files you've worked on
git add filename.js
git add foldername/

# Or add all changed files (be careful!)
git add .

# Commit with a clear message
git commit -m "Add teacher dashboard basic layout"
```

Step 3: Push Your Work to GitHub

```
# Push your changes to your branch
git push origin your-branch-name
```

Commit Message Rules

Good Commit Messages

```
git commit -m "Add QR scanner functionality to student screen"
git commit -m "Fix login button styling issue"
git commit -m "Create teacher dashboard with class cards"
git commit -m "Add SQLite database connection"
git commit -m "Update attendance calculation logic"
```

Bad Commit Messages (Avoid These)

```
git commit -m "changes"
git commit -m "fix"
git commit -m "update"
git commit -m "work done"
```

Format: Start with Action Word

- **Add:** When adding new features
- **Fix:** When fixing bugs
- **Update:** When modifying existing features
- **Remove:** When deleting code

- **Create:** When creating new files/components

Branch Strategy and Workflow

Branch Hierarchy

```
main (production-ready code)
├── development (integration branch)
    ├── krishna-backend
    ├── shambavi-frontend
    ├── rakshitha-design
    ├── ananya-analytics
    ├── hitharth-features
    ├── yashas-testing
    └── nuhan-ui-support
```

Who Works on Which Branch

Krishna (krishna-backend)

Files You Modify:

- `src/database/`
- `src/services/`
- `src/utils/`
- `App.js`
- Package.json (when adding new libraries)

Example Commits:

```
git commit -m "Add SQLite database schema for attendance"
git commit -m "Create QR code generation service"
git commit -m "Add BLE proximity detection logic"
```

Shambavi (shambavi-frontend)

Files You Modify:

- `src/screens/`
- `src/navigation/`
- `src/components/`
- Style files

Example Commits:

```
git commit -m "Create teacher home screen layout"
git commit -m "Add navigation between screens"
git commit -m "Build student dashboard UI"
```

Rakshitha (rakshitha-design)

Files You Modify:

- `src/assets/`
- `src/styles/`
- `src/components/` (styling parts)
- Design documentation

Example Commits:

```
git commit -m "Add app icons and splash screen"
git commit -m "Create design system colors and fonts"
git commit -m "Update button styles with AI-generated design"
```

Ananya (ananya-analytics)

Files You Modify:

- `src/services/AnalyticsService.js`
- `src/screens/teacher/AnalyticsScreen.js`
- `src/components/charts/`
- Analytics-related utils

Example Commits:

```
git commit -m "Add attendance percentage calculation logic"
git commit -m "Create analytics chart components"
git commit -m "Build student recommendation system"
```

Hitharth (hitharth-features)

Files You Modify:

- `src/screens/teacher/` (advanced features)
- `src/services/` (geofencing, notifications)
- `src/components/` (complex components)

Example Commits:

```
git commit -m "Add geofencing service for campus detection"
git commit -m "Create notification scheduling system"
```

```
git commit -m "Build teacher exception handling screen"
```

Yashas (yashas-testing)

Files You Modify:

- `tests/`
- `docs/`
- Bug fix files (any file for testing)

Example Commits:

```
git commit -m "Add QR code functionality tests"
git commit -m "Create testing documentation"
git commit -m "Fix login screen validation bug"
```

Nuhan (nuhan-ui-support)

Files You Modify:

- `src/components/` (simple components)
- `src/screens/` (basic screens)
- Helper UI files

Example Commits:

```
git commit -m "Create loading spinner component"
git commit -m "Add error message screens"
git commit -m "Build basic form components"
```

Weekly Integration Process

Every Friday - Integration Day (Krishna Leads)

Step 1: Everyone Push Final Changes

```
# Make sure all your work is committed and pushed
git add .
git commit -m "Complete week X tasks"
git push origin your-branch-name
```

Step 2: Krishna Merges All Branches

```
# Krishna switches to development
git checkout development

# Merge each person's work
git merge krishna-backend
git merge shambavi-frontend
git merge rakshitha-design
git merge ananya-analytics
git merge hitharth-features
git merge yashas-testing
git merge nuhan-ui-support

# Test the integrated app
npm start

# If everything works, push to development
git push origin development
```

Step 3: Everyone Updates Their Branch

```
# Get the latest integrated code
git checkout your-branch-name
git pull origin development
```

Common Git Commands Reference

Daily Commands

```
# Check current branch
git branch

# Switch to your branch
git checkout your-branch-name

# See what files changed
git status

# See what exactly changed in files
git diff

# Add files to commit
git add filename.js
git add .  # (adds all files)

# Commit changes
git commit -m "Your message here"

# Push to GitHub
```

```
git push origin your-branch-name

# Get latest changes
git pull origin development
```

When You Get Stuck

```
# Undo last commit (keeps changes)
git reset --soft HEAD~1

# Discard all local changes (BE CAREFUL!)
git reset --hard HEAD

# See commit history
git log --oneline

# Create new branch from current location
git checkout -b new-branch-name
```

File Organization Rules

What Goes Where

Krishna's Responsibility

```
src/
├── database/
│   ├── database.js          # Main database connection
│   ├── schema.js            # Database tables
│   └── queries.js           # Database operations
├── services/
│   ├── QRService.js         # QR code generation/scanning
│   ├── BLEService.js        # Bluetooth functionality
│   ├── SessionService.js    # Session management
│   └── AuthService.js       # Authentication logic
└── utils/
    ├── timeUtils.js         # Time/date helpers
    └── cryptoUtils.js       # Security functions
```

Shambavi's Responsibility

```
src/
├── screens/
│   ├── teacher/
│   │   ├── TeacherHome.js
│   │   ├── ClassDashboard.js
│   │   └── SessionControl.js
│   ├── student/
│   │   ├── StudentHome.js
```

```
│   │   ├── QRScanner.js
│   │   └── AttendanceHistory.js
│   └── common/
│       ├── Login.js
│       └── Loading.js
├── navigation/
│   └── AppNavigator.js      # Main navigation setup
└── components/
    ├── ClassCard.js         # Reusable components
    └── StudentCard.js
```

Rakshitha's Responsibility

```
src/
├── assets/
│   ├── icons/               # App icons
│   ├── images/              # Images and logos
│   └── fonts/               # Custom fonts
├── styles/
│   ├── colors.js            # Color palette
│   ├── typography.js        # Font styles
│   ├── commonStyles.js      # Shared styles
│   └── theme.js             # Overall theme
└── components/
    └── styled/              # Styled components
        ├── StyledButton.js
        └── StyledCard.js
```

Conflict Resolution Guide

When Git Says "Conflict"

Step 1: Don't Panic

- Conflicts happen when two people edit the same file
- Krishna will help resolve all conflicts

Step 2: Contact Krishna

- Message in WhatsApp group: "Git conflict in [filename]"
- Don't try to fix it yourself initially
- Share screenshot of error message

Step 3: Learn Conflict Resolution (Advanced)

```
# When you pull and get conflicts
git pull origin development

# Git will show conflict files
# Open the file in VS Code
# You'll see sections like:
<<<<<<< HEAD
Your changes
=======
Other person's changes
>>>>>>> branch-name

# Choose which version to keep or combine both
# Remove the <<<<<<, =======, >>>>>>> lines
# Save the file

# Add the resolved file
git add conflicted-file.js

# Commit the resolution
git commit -m "Resolve merge conflict in filename"
```

GitHub Desktop Alternative (Easier for Beginners)

If Command Line is Difficult

Download GitHub Desktop

1. Go to desktop.github.com
2. Download and install GitHub Desktop
3. Sign in with your GitHub account
4. Clone repository using the app
5. All git commands can be done with buttons

GitHub Desktop Workflow

1. **Clone Repository:** File → Clone Repository
2. **Switch Branch:** Click branch name at top
3. **See Changes:** Changes tab shows modified files
4. **Commit:** Add commit message and click "Commit to branch"
5. **Push:** Click "Push origin" button
6. **Pull:** Click "Fetch origin" then "Pull origin"

Weekly Git Tasks by Person

Week 1 Git Tasks

Krishna

```
# Day 1: Create repository and branches (done once)
# Daily commits for:
git commit -m "Add React Native project setup"
git commit -m "Create SQLite database connection"
git commit -m "Add QR code generation service"
git commit -m "Build session management logic"
```

Shambavi

```
# Daily commits for:
git commit -m "Create basic app navigation"
git commit -m "Add teacher home screen"
git commit -m "Build student dashboard layout"
git commit -m "Add QR scanner screen"
```

Rakshitha

```
# Daily commits for:
git commit -m "Add app icon and splash screen"
git commit -m "Create design system colors"
git commit -m "Add custom fonts and typography"
git commit -m "Build reusable button components"
```

Others

```
# Commit when you complete tasks:
git commit -m "Add loading screen component"
git commit -m "Create error handling screens"
git commit -m "Update project documentation"
```

Emergency Git Commands

When Something Goes Wrong

Undo Last Commit (Keep Changes)

```
git reset --soft HEAD~1
```

Discard All Local Changes

```
git checkout .
```

Start Over from Last Commit

```
git reset --hard HEAD
```

Create Backup Branch Before Big Changes

```
git checkout -b backup-branch-name
git checkout original-branch-name
```

Contact Krishna Immediately If:

- You see "detached HEAD state"
- Git says "fatal error"
- You accidentally deleted important files
- You can't push or pull

Success Checklist for Git

Daily Checklist (Everyone)

- [ ] Started work by pulling latest changes
- [ ] Made commits with clear messages
- [ ] Pushed changes to your branch
- [ ] No conflicts in your branch

Weekly Checklist (Friday)

- [ ] All week's work committed and pushed
- [ ] Branch is up to date with development
- [ ] No merge conflicts
- [ ] App runs successfully on your branch

Monthly Checklist

- [ ] Documentation is updated
- [ ] No unnecessary files in repository
- [ ] All team members can access and contribute
- [ ] Backup of important work exists

Follow this Git guide and you'll never lose code or create problems for the team!